



꿈을 향해 전진하는 개발자 서동규의 이야기

CONTACT

PHONE : 010-6219-8982 | MAIL : LMATARODO@GMAIL.COM

서동규

DONGGYU SSEO (2001.02.11) | 성균관대학교 소프트웨어학과

서동규 2001.02.11. ☎ 010-6219-8982 ✉ lmatarodo@gmail.com

학력사항	2016.03 ~ 2019.02	서울영일고등학교 재학
	2020.03 ~ 2027.02	성균관대학교 소프트웨어학과 재학 전체 학점 3.87/4.5 · 전공 학점 4.03/4.5
개발 프로젝트	2023.06 ~ 2023.07	제 1회 성균관대 자율주행 경진대회 출전
	2025.01 ~ 2025.08	모본(주) 인턴십 - 운전자 보조 시스템 개발
	2025.04 ~ 2025.12	CLM&S 중고차 부품 불량검사 산학 프로젝트
	2025.06 ~ 2025.07	제 3회 성균관대 자율주행 경진대회 출전(팀장)
	2025.12 ~ 2026.12	성균관대 오픈소스 커뮤니티 플랫폼 (SOSD) 백엔드 개발
대외활동 및 경험	2025.01 ~ 2026.08	성균관대 자작자동차 동아리(HEVEN) 자율차 인지팀 부원
	2026.05 ~ 2026.08	수원여고 성균 AI멘토링 봉사활동
	2026.03 ~ 2026.06	발표 연합동아리(웨이브어스) 부원 활동
	2026.08 ~ 2026.12	발표 연합동아리(웨이브어스) 임원진 활동
	2026.03 ~ 2026.12	배드민턴 연합동아리(배라) 부원 활동
어학 및 자격 사항	2025.09.05.	OPIC(영어) IM2
	2026.08.28.	데이터분석준전문가(ADSP)



기술 스택

Backend

JAVA / Spring Boot
Spring Data JPA / Spring Batch
Lombok
Github REST API / GraphQL API

DB

MySQL

Server / Deployment

Docker
CI / CD 자동 배포
Linux

Frontend

React / JavaScript
Vite
Axios

AI / Computer Vision

PyTorch · TensorFlow · OpenCV
CNN · YOLO · DINOv2
t-SNE · Optuna
LLM API · ReAct Agent

Embedded

ROS · ROS2
Zynq SoC · FPGA · Vitis AI · DPU
Arduino · PWM · SPI
PID · Kanayama Control

01. 제1회 성균관대 자율주행 경진대회

차선 인식과 차량 제어를 결합한 자율주행
알고리즘 개발

02. 제3회 성균관대 자율주행 경진대회

FPGA 환경에서 객체 인식과 차량 제어를
통합한 자율주행 시스템 개발

03. 성균관대 오픈소스 커뮤니티 플랫폼 (SOSD)

성균관대 학생들의 오픈소스 활동을 장려하는
플랫폼에서 백엔드 개발 담당

04. 중고차 부품 불량 판별 인공지능 모델

차량 부품의 양품·불량을 자동 판별하는
2-Stage 비전 모델 개발

05. 운전자 보조 시스템

졸음 감지와 차선 이탈 인식을 결합한 실시간
운전자 경고 시스템 개발

유아용 전동차 자율주행 시스템 제 1회 성균관대 자율주행 경진대회



소개

유아용 전동차 플랫폼에 노트북 기반으로
차선 자율주행/자동 주차 기능을 구현하는 대회

기간/인원

2023.06 ~ 2023.07 / 4명

역할

주행 알고리즘 설계 공동 고민, 실차 테스트 및 파라미터 튜닝 반복 수행

성과

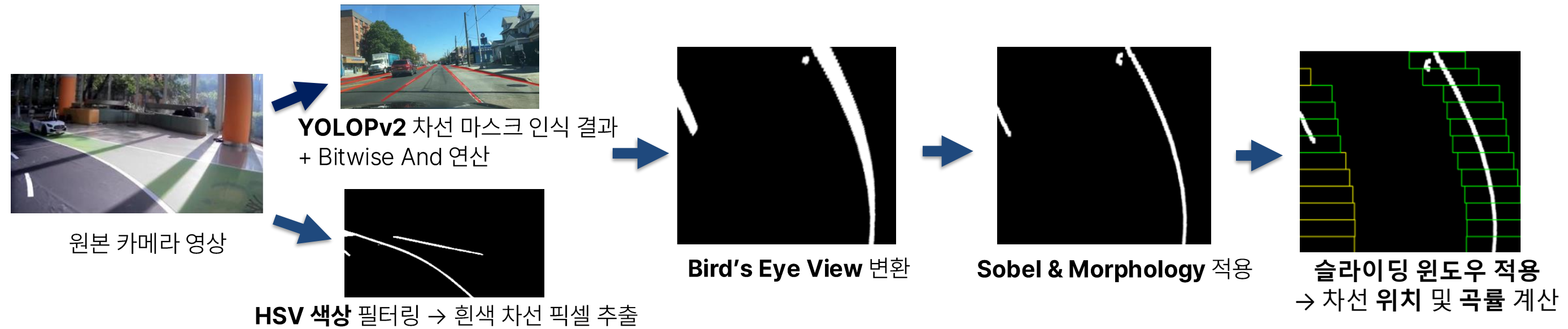
직선 구간 안정성, 곡선 구간 추종 성능 확보
대회 고속주행상 수상
개발 알고리즘 한국정보처리학회(KIPS) 논문 발표 (2023/10)

사용 기술

인지: YOLOv2 · HSV · Bird's Eye View · Sobel Filter · Histogram / Polyfit
제어: PID Control · 직선 및 곡선 구간 적응형 제어 · Arduino

알고리즘 구조도

인지 파이프라인



제어 로직

1. 횡오차 계산

- 목표 차선 중심과 현재 차선 중심의 차이로 횡오차 산출

2. PID 기반 1차 조향값 계산

- 횡오차를 PID 제어기에 입력해 기본 조향값 산출

3. 차선 곡률 계산

- 차선의 기울기 변화량으로 곡률을 계산해 직선·곡선 구간 판별

4. 구간별 가중치 조정

- 곡선 구간: 곡률 보정값 비중 ↑
- 직선 구간: PID 조향값 비중 ↑

5. 최종 조향각 출력

- 두 값을 결합해 조향각을 계산하고 Arduino로 전송

주요 트러블슈팅 및 해결

1. 목표 차선 전환

문제: 오른쪽 차선만 보고 주행하도록 설계했으나, 카메라 각도상 **왼쪽 차선만 보이는 구간**이 존재

해결: 목표 차선을 **왼쪽 차선으로 전환**하고 기준 좌표(mid_standard)를 동적으로 변경

결과 · 왼쪽 차선만 검출되는 구간에서도 목표 차선 유지 가능

```
if lane_changed is True:
    # 차선 상태에 다른 목표 좌표 전환
    if current_lane == 'right':
        current_lane = 'left'
        mid_standard = -39
    else:
        current_lane = 'right'
        mid_standard = 544
```

2. 구간별 제어 가중치

문제: 동일한 제어 가중치 적용 시 직선 구간에서는 조향 진동, 곡선 구간에서는 진입 지연 발생

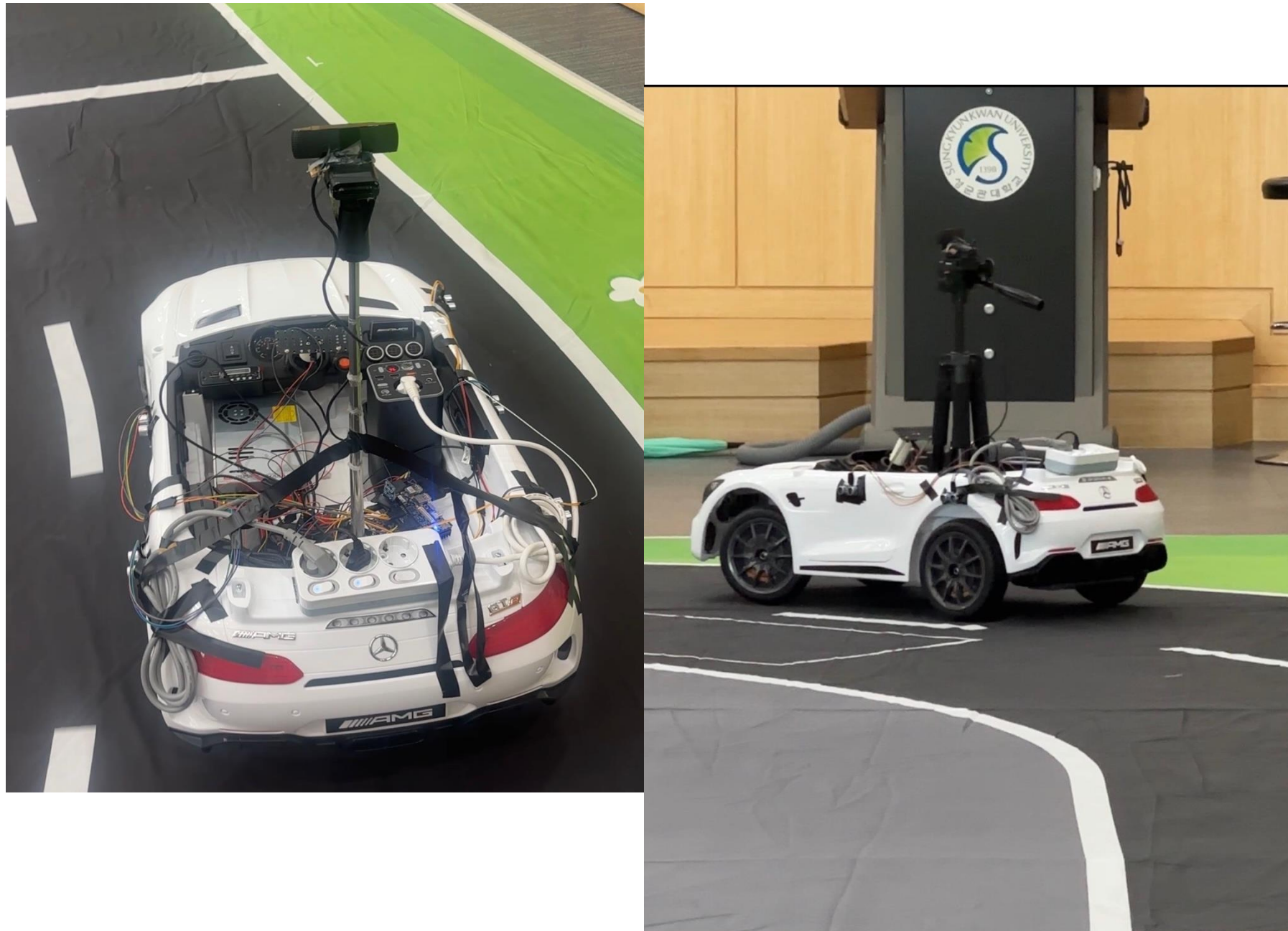
해결: 곡률을 기준으로 구간을 분류하고 횡오차와 곡률 오차의 가중치를 동적으로 조정

결과: 직선 안정성과 곡선 추종성을 동시에 개선

```
# 곡률 기반 조향 가중치 변경
if abs(cal_exactly(right_saving)) < 3:    # 직선 구간
    angle = diff * 0.2 + cal_exactly(right_saving) * 0.2
else:                                     # 곡선 구간
    angle = diff * 0.1 + cal_exactly(right_saving) * 0.5
```

FPGA 기반 자율주행 시스템

제 3회 성균관대 자율주행 경진대회



소개

FPGA 기반 임베디드 환경에서 차선 자율주행과 자동 주차 기능을 구현한 프로젝트

기간 / 인원

2025.06 ~ 2025.07 / 5명

역할

팀장 / 일정 관리 및 진행 총괄 / 주행 및 주차 알고리즘 전체 설계

성과

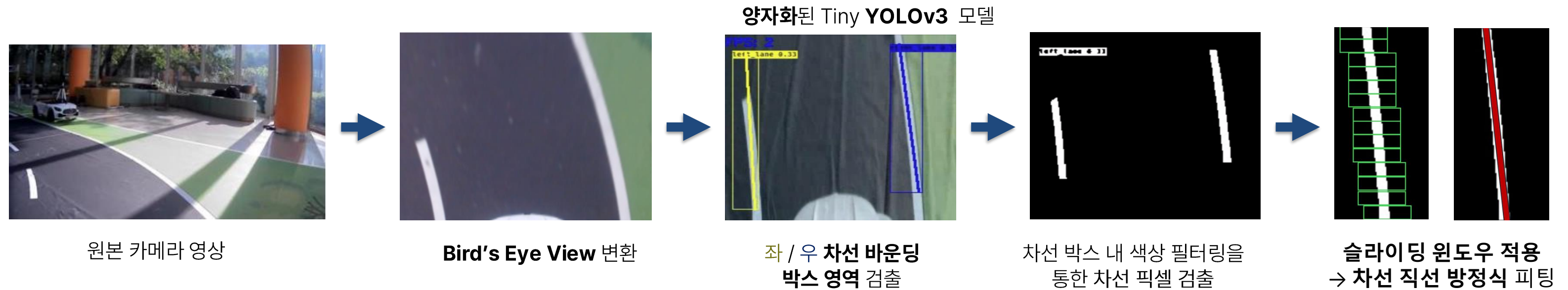
직선 및 곡선 구간에서 안정적인 차선 추종 달성
시각화 로그 시스템 개발로 디버깅 효율 개선
대회 장려상 수상

사용 기술

인지: Tiny YOLOv3 / Bird's Eye View(BEV) / HSV Mask / Sliding Window
제어: FPGA Zynq Soc / Kanayama Control

알고리즘 구조도

인지 파이프라인



제어 로직

1. 횡오차 · 방향오차 계산

- 좌·우 차선의 중심점과 기울기 산출
- 횡오차 e_y , 방향오차 e_θ 계산

2. Kanayama Control

- 목표 속도와 오차 기반으로 제어
- 선속도 v , 각속도 ω 계산

3. 조향각 변환

- 각속도를 Ackermann steering 기반 조향각으로 변환

4. 예외 처리

- 한쪽 차선만 검출 시 반대 차선 위치 추정
- 양쪽 차선 미검출 시 이전 프레임 조향값 유지
- 조향각 급변 방지를 위한 이전값 기반 smoothing

주요 트러블슈팅 및 해결 과정

01. 제한된 튜닝 기간 대응

문제

- 초기 2주가 FPGA·하드웨어 세팅에 소요
- 실차 테스트·파라미터 튜닝 기간이 약 1주로 제한

대응

- 하드웨어 세팅과 병행해 차선 검출·주행 상태 시각화 도구를 사전 개발
- 파라미터별 역할을 정의하고 반복 테스트 수행



결과

하드웨어 완성 직후부터 빠르게 디버깅
제한된 기간 내 주요 주행 파라미터 튜닝 수행

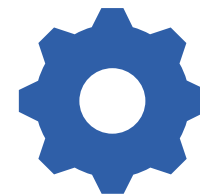
02. 주행 성능 저하 원인의 병렬 검증

문제

- 카메라 각도 변경 후 주행 성능 저하, 원인을 두고 팀 내 의견 분산
 - 딥러닝 모델 성능 저하
 - 조향 모터 결함 가능성
 - 조향 파라미터 미최적화

대응

- 한 가지 원인으로 단정하지 않고 3개 가설을 병렬 검증



조향 모터 점검



모델 재학습



파라미터 재튜닝

결과·회고

대회 종료 전까지 기존 성능을 완전히 복구하지는 못했지만,
복수 가설을 동시에 검증하며 원인을 좁히는 방식의 중요성을 체득

Learned — 감이 아닌 검증 가능한 가설로 원인을 좁히고, 데이터로 판단한다

성균관대 오픈소스 커뮤니티 SOSD

오픈소스 기여 활동 수집 · 분석 및 AI 평가 시스템 개발



소개

성균관대 학생들의 Github 오픈소스 활동을 수집 · 수치화하고,
AI 기반 품질 평가와 피드백을 제공하는 오픈소스 커뮤니티 플랫폼

기간 / 인원

2025.12 ~ 2026.12(진행중) / 2명

역할

백엔드 개발 및 API 설계
GitHub 데이터 수집 구조를 Spring 기반으로 전환
AI 기반 오픈소스 기여 평가 기능 설계·개발

성과

Scrapy → Spring 수집기 전환으로 수집 시간 3일 → 6시간 단축
기여 건수 중심 평가를 AI 기반 품질 평가로 확장
Code-LLM·ReAct Agent 역할 분리로 평가 일관성 확보

사용 기술

Backend: Java · Spring Boot · Spring Data JPA · Spring Batch
DB/API: MySQL · GitHub REST API · GraphQL API
AI: LLM API · ReAct Agent
Infra: Docker · Linux

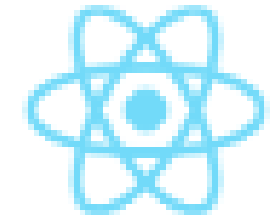
시스템 아키텍처

Backend

django

django
REST
framework

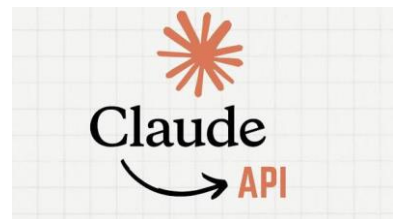
Frontend



React + Vite



GitHub API



Claude API



Github Collector
Scheduler / Batch
Commit Diff API
PR Commit List API



DB
활동 / 평가 / 통계

서비스 API

AI 평가

ReAct 에이전트

LiteLLM

사용량 관리

저장소 · 커밋 · PR · 이슈 정보

REST/GaphQL 요청

통계 · 점수 응답

통계 · 점수 요청

수집 데이터 · 통계 저장

수집 대상 · 기존 데이터 조회

평가 결과 · 사용량 저장

활동 · 기존 평가 조회

JSON 응답

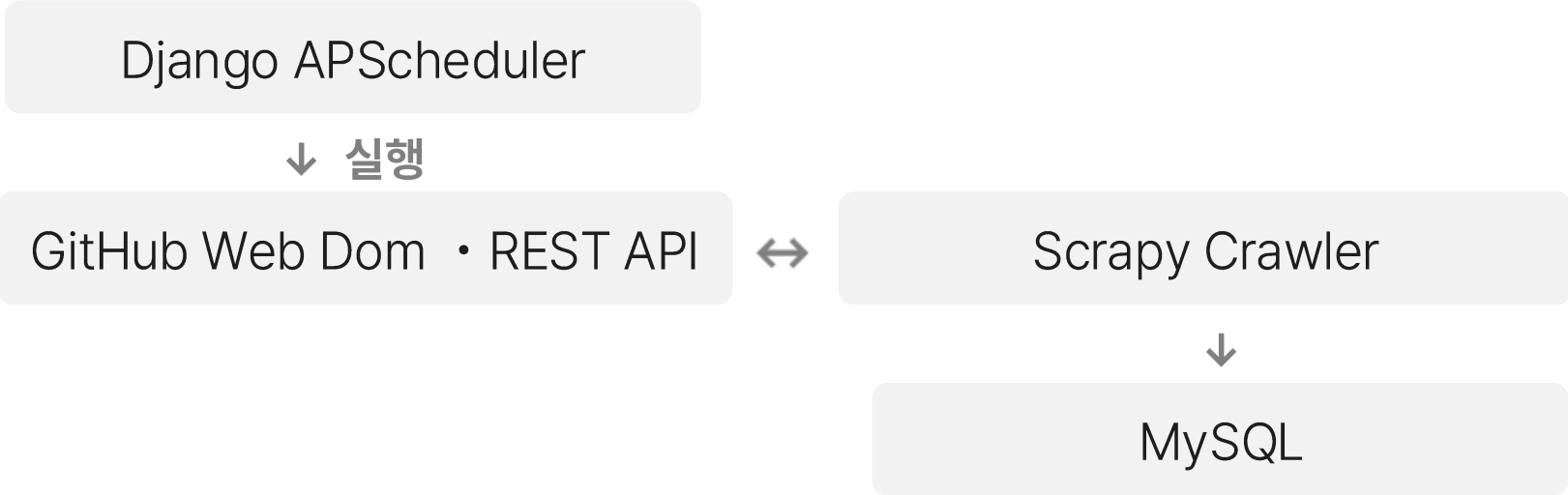
API 요청

프롬프트 요청

판정 · 근거 응답

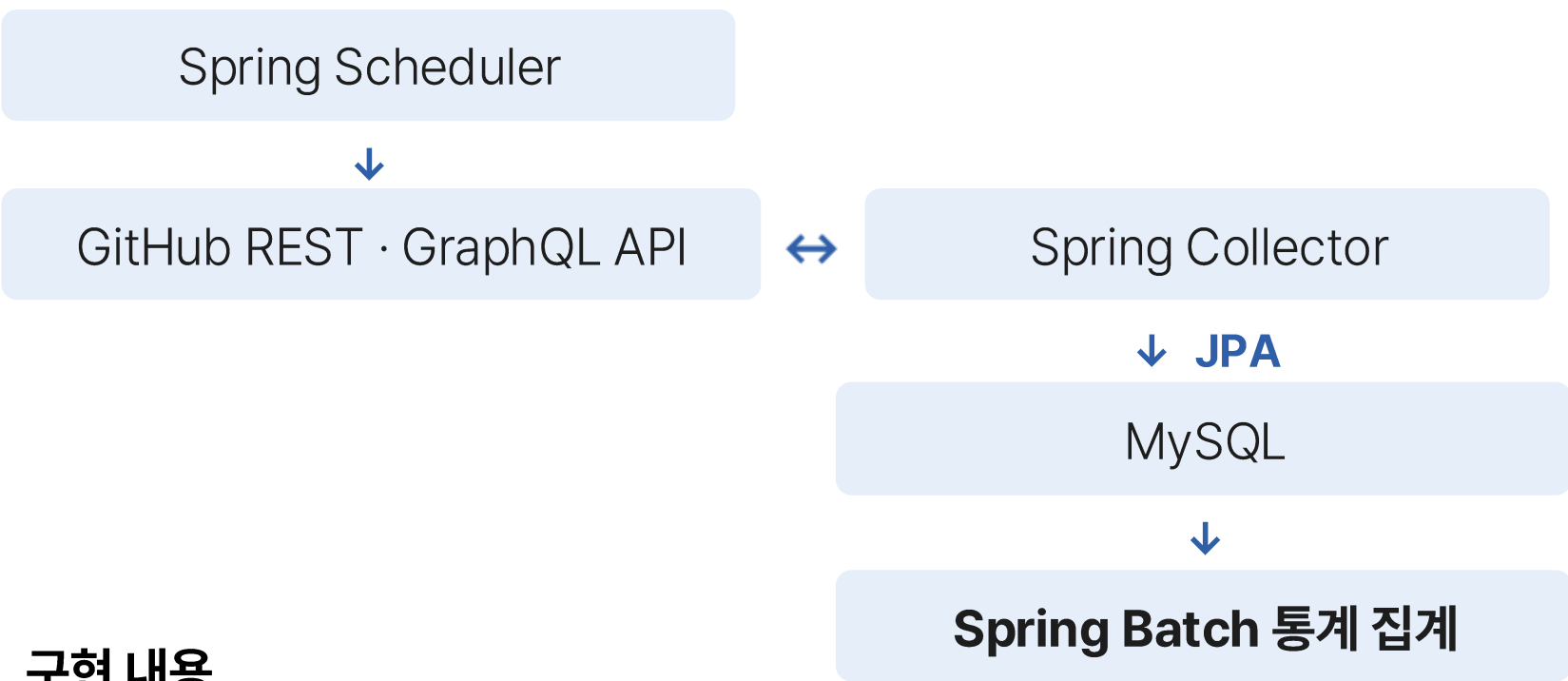
GitHub 활동 수집 구조 개선

Before — Django · Scrapy 기반 수집



- 문제**
- 전체 활동 수집에 약 3일 소요
 - 수집 로직과 서비스 운영 환경 분리
 - 사용자 중심 반복 수집으로 중복 API 호출
 - 수집·집계 로직이 Django·Scrapy에 분산

After — Spring 기반 수집·집계



- 구현 내용**
- GitHub REST · GraphQL API 기반 수집기 구현
 - 사용자 저장소 탐색 후 저장소 중심으로 활동을 수집하는 2단계 구조 적용
 - 증분 커서와 SHA 일괄 조회로 중복 호출 · 조회 최소화
 - Token 자동 전환 및 Pagination 처리
 - Spring Data JPA 기반 활동 데이터 저장
 - Spring Batch 기반 기여 통계 집계

성과 · 전체 수집 및 업데이트 시간 약 3일 → 6시간으로 단축

MySQL VIEW 기반 레거시 호환 레이어 설계

문제: Spring 수집기는 GitHub 계정 · 저장소 · 활동 데이터를 정규화된 테이블로 저장하지만, 기존 Django 서비스는 저장소명과 사용자 정보가 결합된 통합 테이블 형태를 기대

- Spring 정규화 테이블과 Django 레거시 통합 테이블 형태의 데이터 모델 불일치
- 기존 Django 조회 로직 전체를 수정하면 마이그레이션 비용이 큼
- 레거시 형식 테이블을 따로 유지하면 동일 데이터를 이중 저장·동기화해야 함

Before

Scrapy



레거시 통합 테이블

· github_repo_commits · github_repo_stats
· github_issues · github_pulls



Django ORM

After

Spring 정규화 테이블

· github_account
· github_repository
· github_account_repository
· github_commit
· github_issue
· github_pull_request



MySQL VIEW

· v_github_repo_commits
· v_github_repo_stats
· v_github_issues
· v_github_pulls
· v_github_repo_contributor



Django ORM

Spring 테이블 → VIEW: JOIN · 컬럼명 변환으로 레거시 형태 제공

VIEW → Django: managed=False를 적용해 VIEW의 생성 · 변경 책임을 Django Migration과 분리

설계 선택

- 정규화된 원본 스키마 유지
- 레거시 통합 테이블 중복 저장 방지
- 기존 Django ORM 코드 변경 범위 최소화

적용 결과

- 5개 Django 모델을 Spring 기반 VIEW로 전환
- db_table 매핑과 일부 조회 조건만 수정해 기존 로직 재사용
- Spring의 정규화 구조를 유지하면서 Django가 필요로 하는 통합 조회 형태 제공

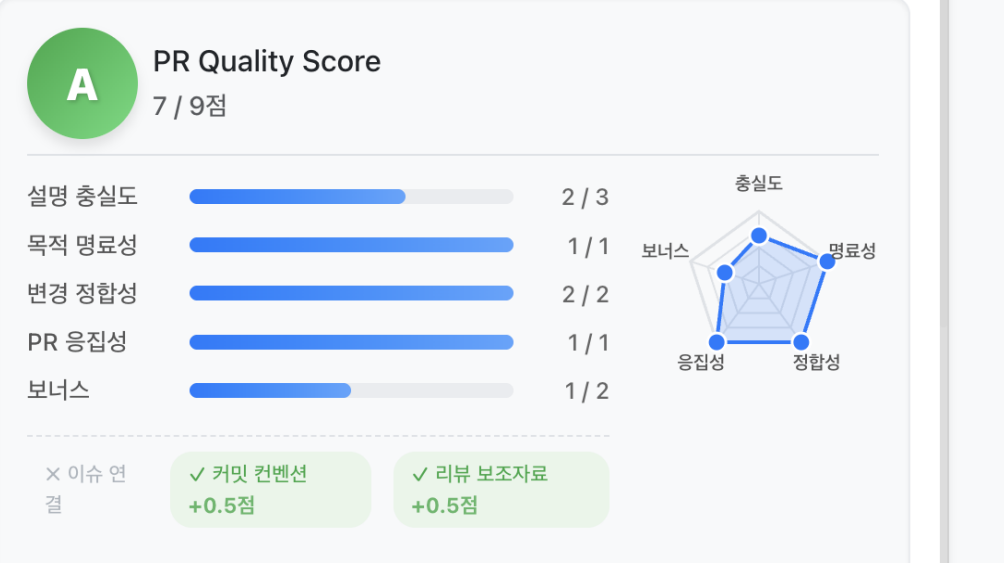
AI 기반 GitHub 기여 평가 시스템 개발

규칙 기반 로직·LLM·ReAct Agent의 역할을 분리해 평가 일관성과 근거성 확보

평가 목적 기여 건수 중심의 평가를 넘어, README·PR·Issue·Commit이 협업 관점에서 충분한 맥락과 검증 근거를 제공하는지 평가하고 구체적인 개선 방향을 제시



1 협업 관점의 점수 계산



2 잘한 점 / 보완할 점 피드백

Strengths (잘한 점)

프로젝트의 목적('AI 기반 개인화 추천과 대화형 검색으로 취향에 꼭 맞는 영화를 찾아보세요')이 명확하고, 맞춤 추천, 캐릭터 채팅, AI 추천 채팅 등 주요 기능들이 구체적으로 명시되어 있습니다.

의존성 설치('pnpm install')와 개발 서버 실행('pnpm dev') 명령어가 명확하게 제시되어 있으며, 환경 변수 설정 및 데이터 준비 과정도 단계별로 상세히 안내하고 있습니다.

마크다운 헤딩 구조가 체계적으로 구성되어 있고, 코드 블록과 표, 목록 등이 일관되게 활용되어 전체 문서의 흐름과 가독성이 우수합니다.

Improvements (보완할 점)

프로젝트의 주요 기능과 기술 스택은 명확하지만, 이 애플리케이션이 어떤 사용자를 대상으로 하는지, 언제 어떤 상황에서 사용하면 좋을지에 대한 구체적인 설명이 부족합니다.

패키지 설치와 실행 명령어는 제시되어 있으나, 각 명령어 실행 후 터미널에 출력되는 예상 결과나 기대하는 동작에 대한 설명이 없어 사용자가 정상 실행 여부를 판단하기 어렵습니다.

설계 원칙 | 규칙 기반 판정은 코드로, 문맥 기반 판단은 AI로 역할을 분리하고, 최종 점수는 코드로 일관되게 계산

AI 평가의 일관성과 신뢰성 확보

규칙 기반 판정 · 문맥 기반 판정 · 문장화 · 에이전트 탐색의 역할을 분리

핵심 메시지 판정 책임을 Code와 LLM으로 나누고, 판정 이후의 문장화·탐색 단계까지 분리해 **실행 간 흔들림 없이 일관된 평가 결과를 확보**

① 판정 성격에 따라 Code와 LLM 역할 분리

Code

- 명령어·Heading 존재 여부
- 컨벤션·형식 준수
- 자동 생성물 필터링
- 고정 배점 계산

LLM

- 기능 설명의 충분성
- Issue·PR 유형 분류
- 변경 사유의 적절성
- 문맥 기반 판단



충족 여부 + 판단 이유



코드 기반 점수 계산

규칙으로 확인 가능한 항목은 Code가, 문맥 이해가 필요한 항목은 LLM이 판정.
LLM은 충족 여부와 판단 이유만 반환하고, 최종 점수는 코드에서 계산

② 판정과 문장화 분리

Code · LLM 판정



충족 여부 · 판단
이유 · 근거



문장화 LLM



칭찬 · 개선점 · 실행
조언

확정된 충족 여부 · 판정 이유 · 근거만 문장화 LLM에 전달해 피드백이 평가 결과와 어긋나는 것을 방지
판정은 먼저 고정하고, 문장화는 그 결과를 설명하는 역할로 제한

③ PR 정합성 · 응집성 평가 ReAct Agent의 탐색·판정 일관성 확보

초기 문제

코드의 구현 결함을 PR 설명 불일치로 오판
릴리즈 PR의 여러 기능을 하나의 목적으로 해석

개선

Golden Set 구축: 실제 오판 · 경계 사례를 고정해 프롬프트 변경 시 회귀 검증
판정 경계 명확화 – 정답, 반례, 금지 기준, 애매한 경우의 처리 원칙을 프롬프트에 명시
행동 선택과 최종 판정 분리: 필요한 코드 diff를 수집한 뒤 별도 호출에서 최종 판정
확인한 patch만 근거로 사용: 확인한 제목, source patch만 사용하고 미확인 내용은 추측 금지

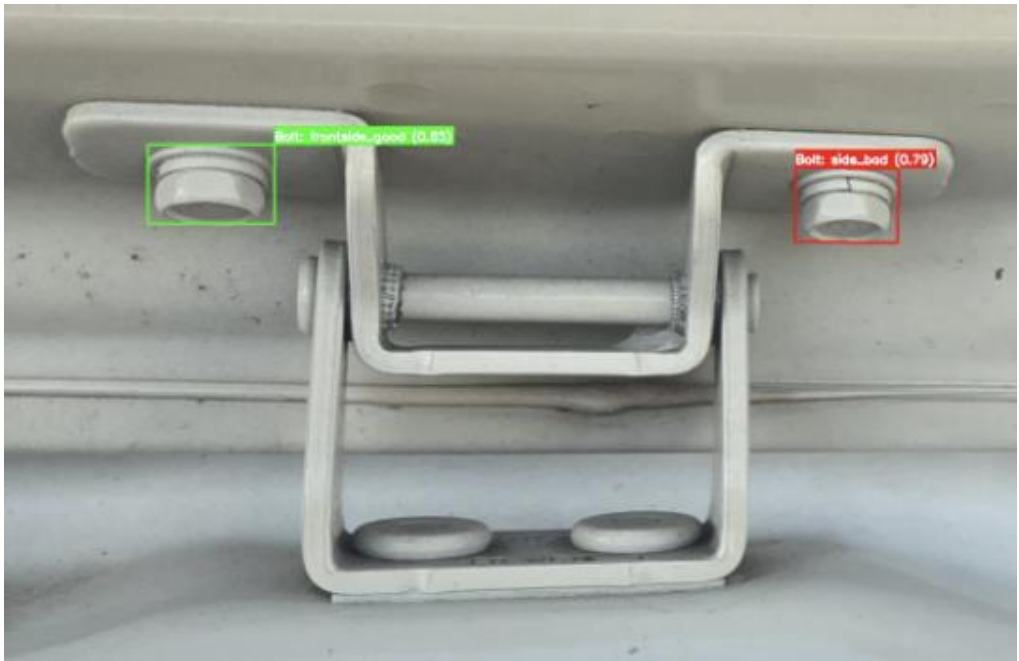
결과

경계 사례에서도 탐색 경로와 최종 판정의 일관성 강화

설계 원칙 | 규칙 기반 판정은 코드로, 문맥 기반 판단은 AI로 역할을 분리하고, 최종 점수는 코드로 일관되게 계산

AI기반 중고차 부위별 교환 여부 판정 기술 개발

성균관대학교-CLM&S 산학 협력 프로젝트



소개

중고 자동차의 앞도어 · 트렁크리드 · 후드 · 프론트펜더 부위에서 실링 · 볼트 불량을 자동 판별하는 AI 품질 검사 시스템 개발

기간 / 인원

2025.04 ~ 2025.12 / 3명

역할

프로젝트 기획 · 개발 / 모델 성능 평가 / 볼트 양 · 불량 분류 모델 개발

성과

단일 YOLO 구조 → 2-Stage 구조로 전환, 부위별 양 · 불량 분류 정확도 85~96% 달성
DINOv2 + 단계적 Fine-tuning으로 학습 편향을 완화하고 분류 안정성 개선
T-SNE · Optuna · YOLO Evolve를 활용해 모델 해석과 하이퍼파라미터 탐색 체계화

사용 기술

Detection: YOLOv11 · OBB

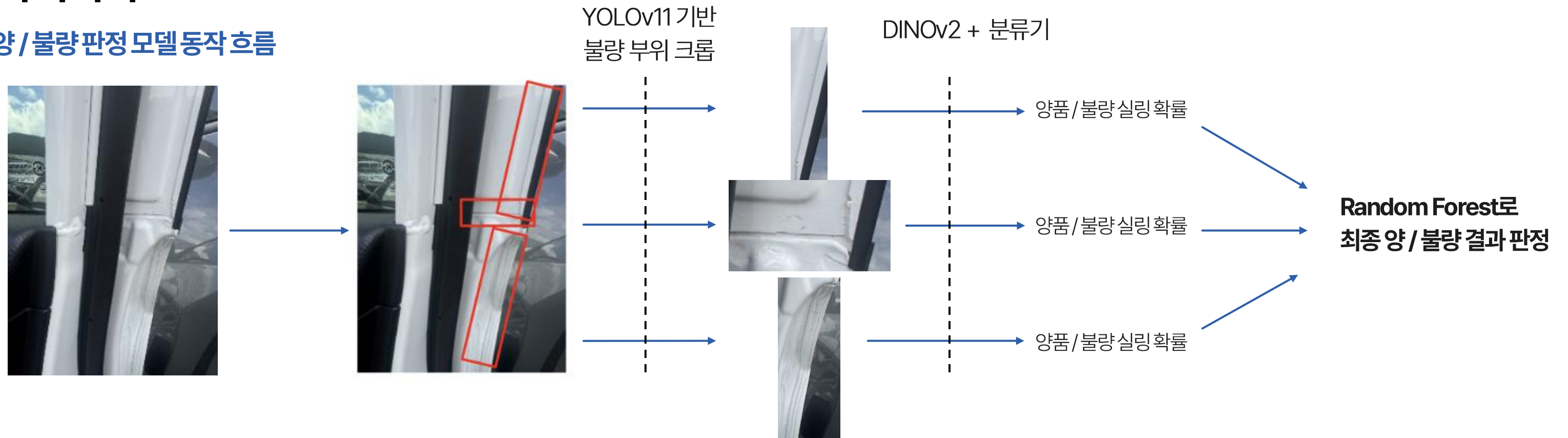
Classification: ResNet · DINOv2 · Random Forest

Optimization: Optuna · YOLO Evolve

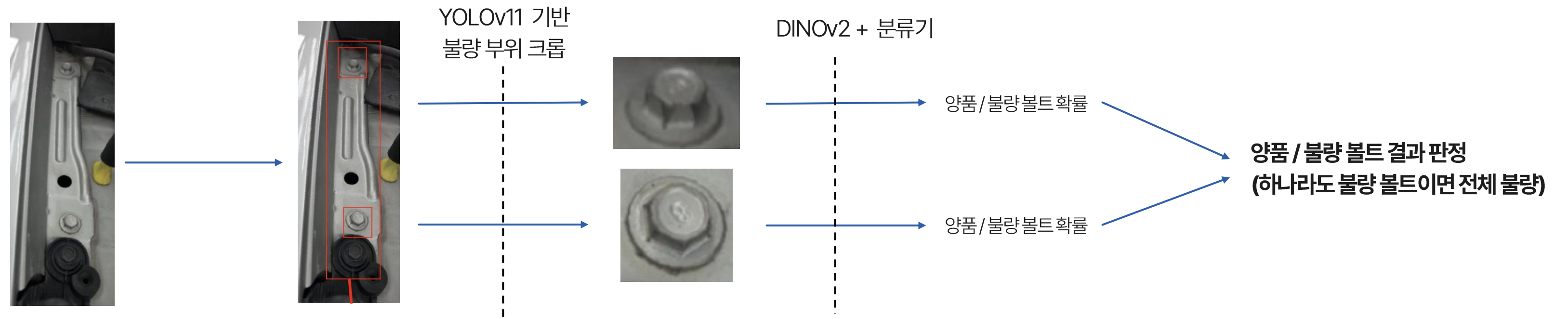
Analysis: t-SNE

시스템 아키텍처

앞도어 실링 양 / 불량 판정 모델 동작 흐름



볼트 양 / 불량 판정 모델 동작 흐름



모델 성능 개선 및 데이터 한계 분석

구조·하이퍼파라미터·특징 공간을 개선하고, 판별 자체가 불가능한 데이터 조건까지 검증

① 구조 개선·YOLO 단일 판정 → 2-Stage

기존 YOLO → 양/불량 판정

개선 YOLOv11 + OBB → Crop → DINOv2 분류

탐지와 분류를 분리해 각 단계의 역할을 명확히 하고 성능 개선

② 학습·탐색 개선

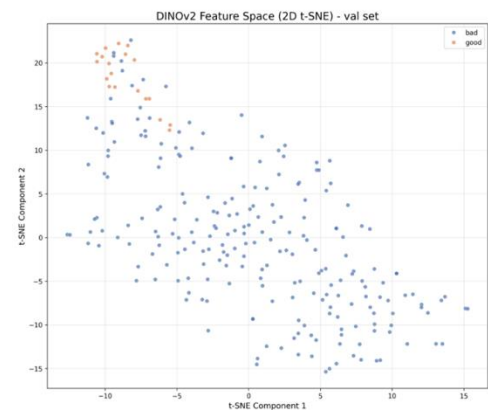
Grid Search → Optuna + YOLO Evolve

ResNet → DINOv2

수동 탐색의 한계를 자동 탐색으로 전환

사전학습 특징을 활용해 분류 성능 및 학습 안정성 개선

③ t-SNE 기반 특징 공간 검증



t-SNE 기반 특징 공간 검증

양품·불량 임베딩의 분리 정도를 시각화해 정확도 외에도 모델이 학습한 특징을 확인

④ 판별 불가능한 데이터 조건 발견



측면/정측면
까짐·부식 관측 가능

정면
핵심 불량 특징 관측 불가

정면 촬영본에서는 불량 판단의 핵심 근거가 보이지 않아, 모델 성능 문제가 아니라 입력 데이터 자체의 판별 가능성 문제로 판단

대응 및 현장 적용 제안

- 정면 볼트 이미지를 학습 데이터에서 전량 제외
- 판별 가능 / 불가능 정면의 분리 학습은 실사용 혼재 위험으로 기각
- 작업자가 정측면·측면 촬영을 유도하는 UI 방향 제안

핵심 | 모델을 더 학습시키기 전에, 데이터가 실제로 판별 근거를 담고 있는지부터 확인

실시간 운전자 보조 시스템 개발 모본(주) 인턴십 산학 협력 프로젝트



Real-Time Drowsiness and Lane Departure Warning System Using Deep Learning and Embedded Systems

Donggyu Seo
Dept. of Computer Science
and Engineering
Sungkyunkwan University
Suwon-si, Republic of Korea
lmatarodo@gmail.com

Jiyoung Moon
Dept. of Mechanical
Engineering
Sungkyunkwan University
Suwon-si, Republic of Korea
rain527@g.skku.edu

Young Hoon Suh
Dept. of Electrical and
Computer Engineering
Sungkyunkwan University
Suwon-si, Republic of Korea
dudgns0407@g.skku.edu

Jae Wook Jeon
Dept. of Electrical and
Computer Engineering
Sungkyunkwan University
Suwon-si, Republic of Korea
jwjeon@skku.edu

Abstract— This study proposes the implementation of a driver drowsiness detection and lane departure warning system. The main components of the system include a camera, PC, Arduino, buzzer, and LED. Real-time data of both the driver and the road are collected through the camera. Lane departure is determined using the YOLOPv2 deep learning model combined with post-processing techniques, while driver drowsiness is assessed by employing Dlib-based facial landmark detection in conjunction with the Eye Aspect Ratio (EAR) method. The results from both detection modules are integrated through the main processing unit, which comprehensively evaluates the driver's drowsiness state and lane departure status. When a risk is identified, the Arduino activates the buzzer and LED to deliver immediate alerts to the driver. The system operates in real time (~60 FPS) and is designed to enhance driving safety by mitigating risks associated with drowsiness and unintended lane departure.

Keywords — Driver drowsiness detection, Lane departure warning, YOLOPv2, Dlib, Eye Aspect Ratio (EAR), Real-time monitoring, Arduino

I. INTRODUCTION

Modern automobiles have evolved beyond mere transportation, developing into intelligent systems that offer drivers various forms of convenience and safety. In particular, technologies that monitor the driver's condition in real-time and detect potential hazards play a crucial role in preventing traffic accidents.

According to statistics from the National Police Agency, a total of 10,765 traffic accidents were caused by drowsy driving between 2019 and 2023, averaging 5.9 incidents per day [1]. As drowsy driving is one of the leading causes of traffic accidents, the need for real-time monitoring and warning systems to prevent it is increasing.

In this study, we implement a drowsiness and lane departure detection and warning system utilizing deep learning and an embedded system. By integrating these functions, the system provides a warning via an audible alarm

and an LED light, controlled by an Arduino, when the driver is determined to be drowsy.

II. THEORETICAL BACKGROUND AND SYSTEM ARCHITECTURE

Driver state detection is performed through an in-vehicle camera using Dlib-based facial landmark detection and Eye Aspect Ratio (EAR) analysis. Based on the EAR value, if eye closure persists for a certain duration, the driver is determined to be in a drowsy state. The open-source Dlib library provides pre-trained models that can be applied without additional training, making it highly practical. Moreover, compared to OpenCV-based implementations, Dlib's facial landmark regression is reported to run extremely fast, achieving landmark prediction in about 1 ms per face (~1000 FPS), which makes it well-suited for real-time driver monitoring systems [3], [4]. Thus, Dlib was adopted in the development of the driver state detection system [5].

Lane departure detection is carried out by processing forward-facing driving video input through the YOLOPv2 model to detect lane markings. YOLOPv5 Segmentation detected lanes in real time (48 FPS, 89% accuracy, high mAP), showing its practicality [6]. The number of lane pixels within the Region of Interest (ROI) is analyzed, and if the count falls below a threshold, the system determines that a lane departure has occurred.

Both drowsiness detection and lane departure detection run independently through multiprocessing. In the drowsiness detection process, if the driver's eyes remain closed for a specified period, the status is transmitted to the main process [7]. Similarly, the lane departure detection process sends a signal to the main process when a departure is detected. The main process integrates these results, and if both drowsiness and lane departure occur simultaneously, it activates a buzzer

소개

운전자 졸음과 차선 이탈을 동시에 감지하고, 위험 상황에서 즉시 경고하는 ADAS 시스템 구현

기간 / 인원

2025.01 ~ 2025.07 / 2명

역할

프로젝트 기획 · 개발 / 시스템 통합 / 논문 작성
졸음 감지 · 차선 이탈 인식 모듈 및 Arduino 경고 시스템 구현

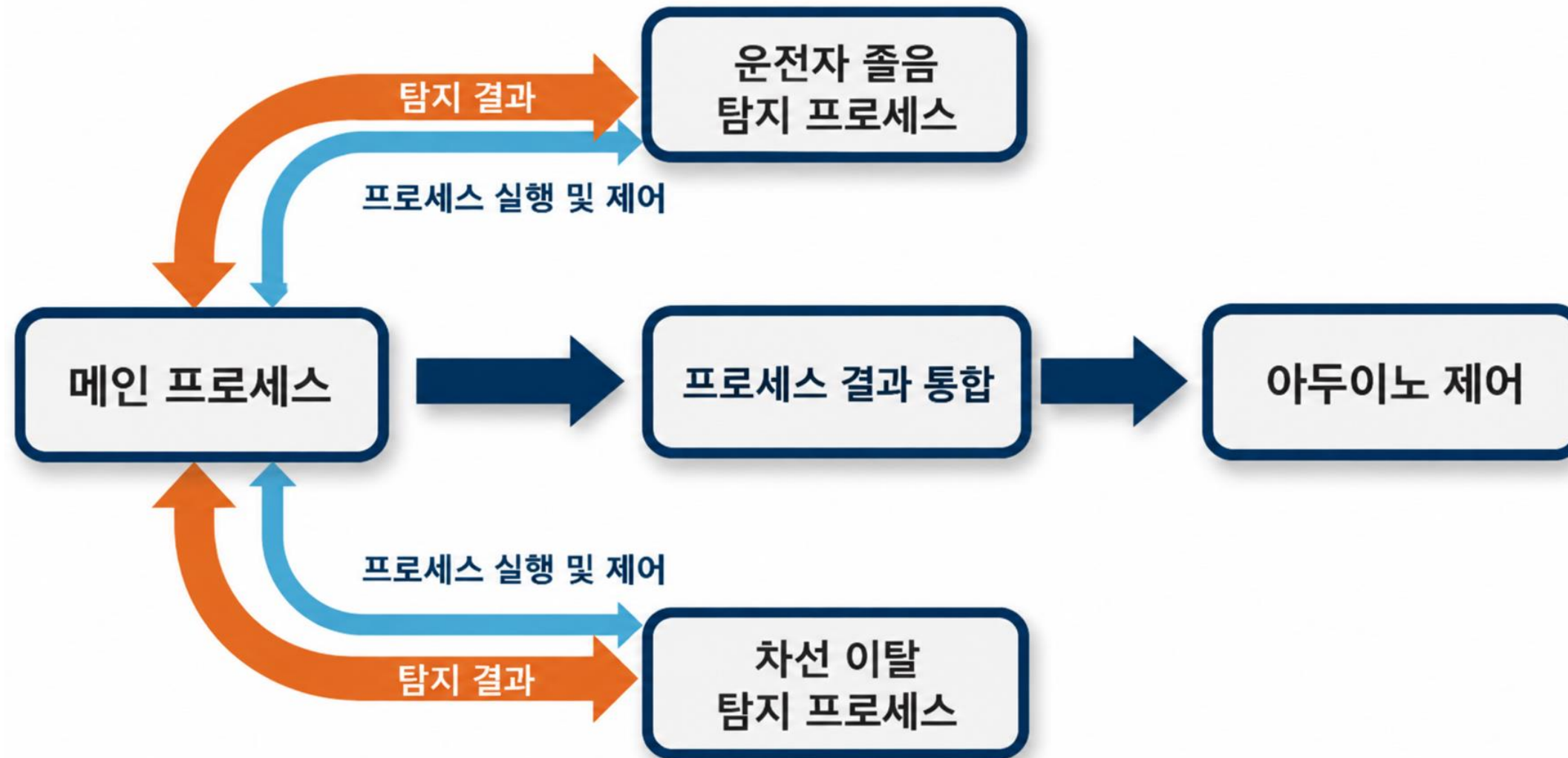
성과

졸음 감지와 차선 이탈 감지를 결합한 실시간 ADAS 시스템 구현
멀티 프로세싱 기반 병렬 처리로 약 60FPS 실시간성 확보
두 위험 조건을 함께 고려하는 조건부 경고 로직으로 불필요한 경고 최소화
IEEE ICCE-Asia 2025 제 1저자 논문 제출

사용 기술

Perception: Dlib HOG · 68 Facial Landmark · YOLOPv2 · OpenCV
Decision: EAR · Dynamic Threshold · BEV · ROI
System: Python Multiprocessing · Arduino Seerial

시스템 아키텍처



설계 포인트

01 독립 프로세스 실행

졸음 감지와 차선 이탈 감지를 멀티프로세싱으로 병렬 처리

02 통합 판단

메인 프로세스가 두 감지 결과를 수집해 최종 위험 상태 결정

03 조건부 경고

졸음과 차선 이탈이 함께 발생할 때 Red LED와 Buzzer를 작동시켜 불필요한 경고 최소화

≈ 60 FPS

두 감지 모듈을 동시에 실행하면서 실시간 처리 성능 확보

핵심 알고리즘 및 검증 결과

졸음 감지와 차선 이탈 감지를 병렬 처리하고, 두 조건이 동시에 충족될 때만 경고

Driver Drowsiness Detection · 졸음 감지

- 1. Light Preprocessing
- 2. Dlib HOG 얼굴 검출
- 3. 68 Facial Landmark 추출
- 4. EAR 계산

5. 동적 임계값 + 5초 지속 조건으로 졸음 확정

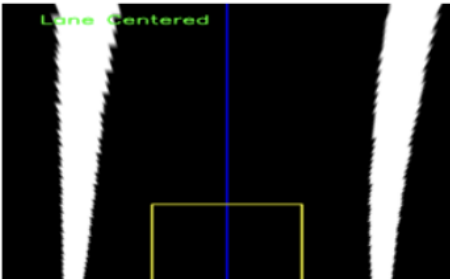
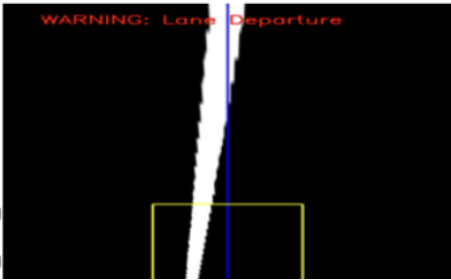


EAR 124.81 (눈 감음) · 305.46 (눈 뜸)

Lane Departure Detection · 차선 이탈 감지

- 1. YOLOPv2 Lane Segmentation
- 2. Binary Mask 생성
- 3. BEV 변환
- 4. ROI 내 차선 픽셀 분석

5. 0.5초 이상 이탈 지속 시 차선 이탈 확정



좌: 차선 이탈 감지 · 우: 정상 주행

졸음 + 차선 이탈



Conditional Alert



Red LED + Buzzer

- 두 위험 조건 동시 충족 → Red LED + Buzzer
- 정상 차선 변경으로 판단 → Green LED

≈ 60 FPS

실시간 처리 성능 확보

Conditional Alert

졸음 + 차선 이탈 동시 조건 기반 경고로 불필요한 경고 최소화

IEEE ICCE-Asia 2025

제1저자 논문 제출